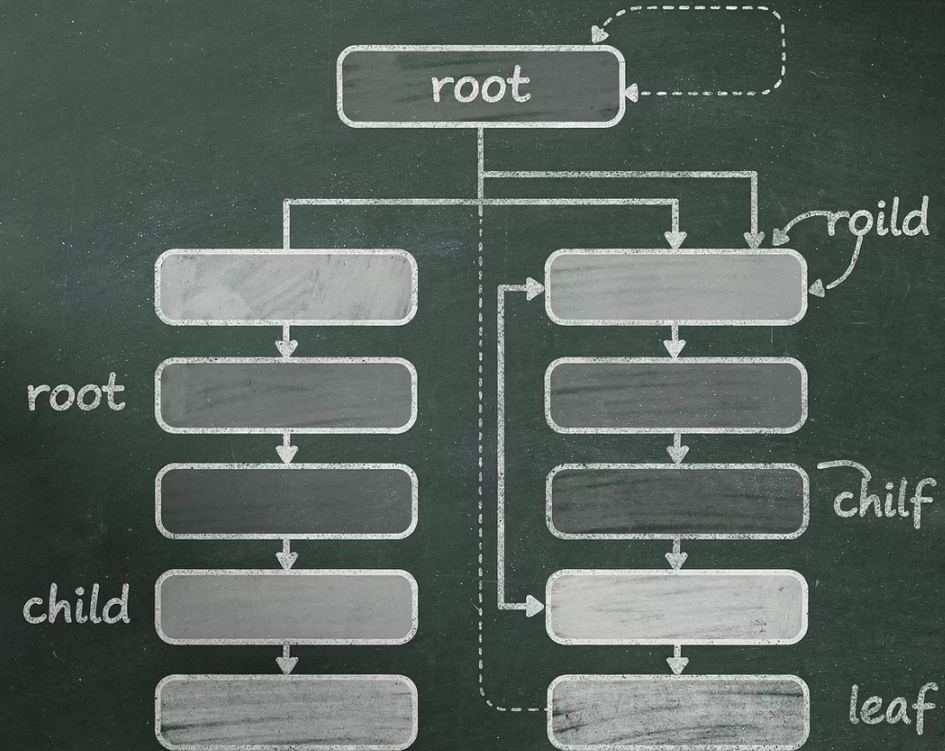


Heap Data Storage



Estruturas de Armazenamento de Dados

Um banco de dados é armazenado como uma coleção de **arquivos**. Cada arquivo é uma sequência de **registros**, e cada registro é composto por uma sequência de **campos**.

Abordagem Básica

A implementação mais simples assume que o tamanho do registro é fixo. Neste modelo, cada arquivo contém registros de apenas um tipo específico, e diferentes arquivos são usados para diferentes relações.

Este é o caso mais fácil de implementar, embora estruturas de comprimento variável sejam consideradas posteriormente. Uma suposição importante é que os registros são menores que um bloco de disco.

Abordagem Simples

Armazena o registro i começando no byte $n \times (i - 1)$, onde n é o tamanho de cada registro.

Acesso Direto

O acesso é simples e direto, permitindo localização rápida através de cálculos matemáticos.

Desafio

Registros podem cruzar limites de blocos, causando ineficiências de I/O.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

 **Modificação Importante:** Para melhorar a eficiência, não permitimos que registros cruzem limites de blocos.

Múltiplos Tipos de Registro

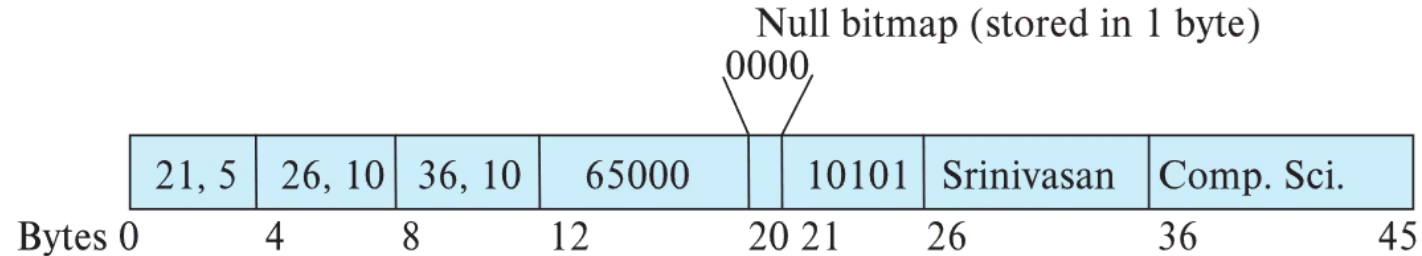
Armazenamento de diferentes tipos de registros em um único arquivo para otimizar o espaço.

Campos de Tamanho Variável

Permitem comprimentos variáveis para um ou mais campos, como strings (**varchar**).

Campos Repetidos

Tipos de registro que permitem campos repetidos, usados em modelos de dados mais antigos.



Estratégia de Armazenamento

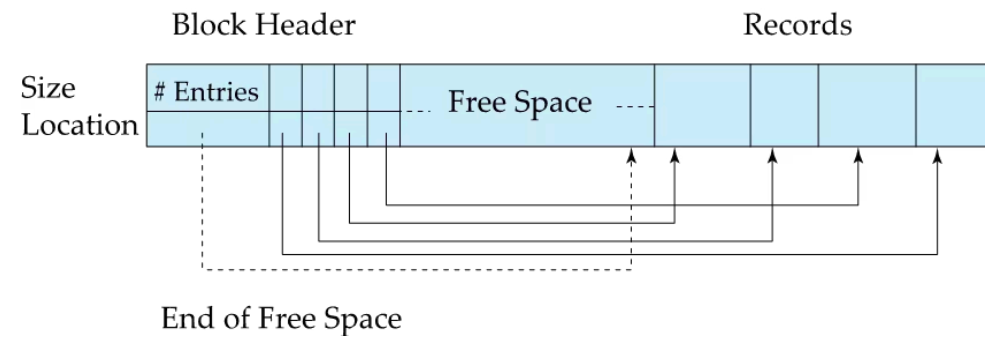
- Atributos armazenados em ordem sequencial
- Atributos de comprimento variável representados por (offset, comprimento) de tamanho fixo
- Dados reais armazenados após todos os atributos de comprimento fixo
- Valores nulos representados por bitmap de valores nulos

A **página com slots** permite gerenciamento eficiente de registros de comprimento variável dentro de um bloco.

Componentes do Cabeçalho

- Número de entradas de registro no bloco
- Posição do fim do espaço livre
- Localização e tamanho de cada registro

Registros podem ser movidos para mantê-los contíguos. A entrada no cabeçalho é atualizada quando isso ocorre.



❏ **Importante:** Ponteiros não devem apontar diretamente para o registro, mas para a entrada no cabeçalho, permitindo reorganização interna.

Tipos como **BLOB** (Binary Large Object) e **CLOB** (Character Large Object) apresentam desafios especiais, pois registros devem ser menores que páginas.

Sistema de Arquivos

Armazenar como arquivos no sistema de arquivos do SO

Gerenciamento pelo BD

Armazenar como arquivos gerenciados diretamente pelo banco de dados

Fragmentação

Dividir em pedaços e armazenar em múltiplas tuplas em uma relação separada

 **Exemplo:** PostgreSQL utiliza TOAST (The Oversized-Attribute Storage Technique) para gerenciar objetos grandes de forma eficiente.

Existem diversas estratégias para organizar registros dentro de arquivos, cada uma com vantagens específicas dependendo dos padrões de acesso aos dados.

Heap (Monte)

Registros colocados em qualquer lugar onde houver espaço. Simples, sem ordem específica.

Sequential (Sequencial)

Registros armazenados em ordem sequencial com base na chave de busca.

Clustering Multitabela

Registros de várias relações no mesmo arquivo, minimizando I/O ao manter dados relacionados no mesmo bloco.

B+-tree

Mantém armazenamento ordenado mesmo com inserções e exclusões. Detalhes na aula de Indexação.

Hashing

Função hash calculada na chave de busca determina o bloco destino do registro. Detalhes na aula de Indexação.

Organização Heap

Registros podem ser inseridos em qualquer lugar com espaço livre. Para novas inserções, é necessário localizar esse espaço eficientemente — para isso existe o **Free-Space Map**: um array com uma entrada por bloco, registrando a fração livre de cada um.

Nível Primário

3 bits por bloco indicam a fração livre (valor ÷ 8).

Nível Secundário

Cada entrada armazena o máximo de 4 entradas do primeiro nível.

Persistência

Mapa gravado periodicamente; valores desatualizados são detectados e corrigidos.

Quando Usar

Adequado para processamento sequencial de todo o arquivo, como relatórios completos ou operações em lote.

Ordem Garantida

Registros mantidos em sequência lógica

Leitura Sequencial

Acesso rápido para varreduras completas

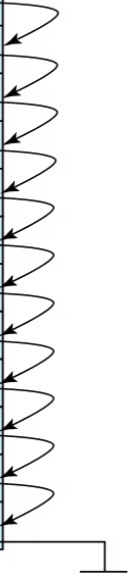
Busca por Intervalo

Eficiente para consultas de faixa de valores

Características

Registros ordenados por uma **chave de busca**, permitindo acesso eficiente a intervalos de dados.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Armazene várias relações em um único arquivo usando **clustering multitabela** — útil quando há relacionamentos frequentes entre tabelas.

Relação Department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

Informações sobre departamentos

Relação Instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Informações sobre instrutores

Clustering Combinado

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Department e Instructor juntos

❏ **Benefício Principal:** Reduz operações de I/O ao manter registros relacionados fisicamente próximos, melhorando o desempenho de joins.

Particionamento de tabelas permite que registros em uma relação sejam divididos em relações menores que são armazenadas separadamente, oferecendo benefícios significativos de desempenho e gerenciamento.

Exemplo Prático

A relação *transaction* pode ser particionada em *transaction_2018*, *transaction_2019*, *transaction_2020*, etc.

Consultas Inteligentes

Consultas em *transaction* acessam todas as partições, a menos que haja uma seleção como *year=2019* — então apenas uma partição é necessária.

Redução de Custos

Reduz custos de operações como gerenciamento de espaço livre, tornando o sistema mais eficiente.

Armazenamento Flexível

Diferentes partições podem ser armazenadas em diferentes dispositivos para otimizar desempenho.

Otimização por Idade

Transações do ano atual em SSD; anos anteriores em disco magnético mais lento e econômico.

Dicionário de Dados

O **Dicionário de Dados** (ou **catálogo do sistema**) armazena **metadados** — dados sobre dados — mantendo informações essenciais sobre toda a estrutura do banco de dados.

Informações sobre Relações

- Nomes das relações
- Nomes, tipos e comprimentos dos atributos
- Nomes e definições de views
- Restrições de integridade

Informações de Usuário

- Dados de usuários e contabilidade
- Senhas e permissões
- Controles de acesso

Dados Estatísticos

- Número de tuplas por relação
- Distribuição dos dados
- Otimização de consultas

Organização Física

- Como a relação é armazenada (sequencial/hash/etc.)
- Localização física da relação
- Informações sobre índices (ver aula de Indexação)

Armazenamento em Disco

Metadados armazenados usando representação relacional, aproveitando as mesmas estruturas dos dados regulares.

Estruturas em Memória

Estruturas especializadas para acesso eficiente em memória, otimizando consultas frequentes ao catálogo.

Persistência

Metadados armazenados como tabelas relacionais em disco

Cache

Estruturas especializadas mantidas em memória principal

Otimização

Acesso rápido através de índices e cache inteligente

Os blocos são unidades de alocação de armazenamento e transferência de dados. O sistema de banco de dados busca minimizar o número de transferências de blocos entre disco e memória.

Estratégia de Otimização

Reduz acessos ao disco mantendo o maior número possível de blocos na memória principal.

Buffer

Porção da memória principal que armazena cópias de blocos de disco, funcionando como cache de alta velocidade.

Gerenciador de Buffer

Subsistema responsável por alocar espaço de buffer e implementar políticas de substituição de blocos.

Os programas chamam o gerenciador de buffer quando precisam de um bloco do disco.

Verificação do Buffer

Se o bloco já estiver no buffer, o endereço na memória principal é retornado imediatamente.

Alocação de Espaço

Se o bloco não estiver no buffer, o gerenciador aloca espaço para ele.

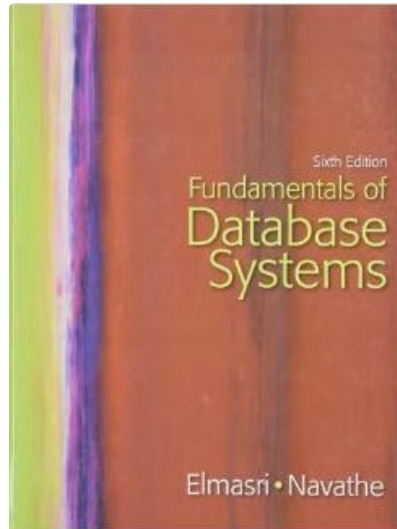
Substituição de Bloco

Outro bloco é descartado se necessário. Blocos modificados são gravados de volta ao disco antes de serem substituídos.

Leitura e Retorno

O bloco é lido do disco para o buffer e o endereço é retornado ao solicitante.

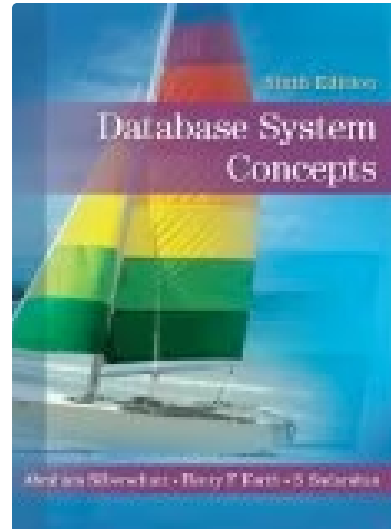
 **Otimização:** Blocos substituídos são gravados no disco apenas se foram modificados, reduzindo operações de I/O desnecessárias.



Elmasri & Navathe

Fundamentals of Database Systems

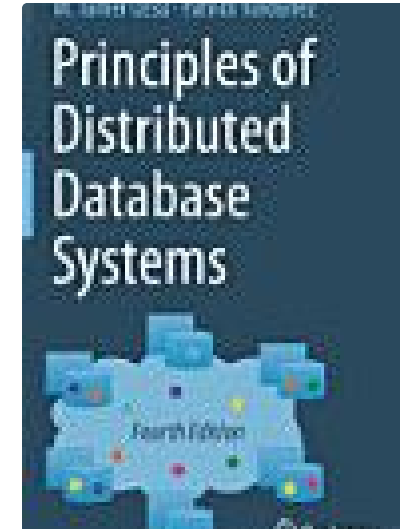
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsü & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019